S08 Bitácora : Saga Circuit Breaker

===> 1. El problema: dinero en el aire

Saldos iniciales

```
curl -i http://localhost:8080/api/cuentas/0021800000000000001
curl -i http://localhost:8080/api/cuentas/0021800000000000002

HTTP/1.1 200 OK
Content-Type: application/json

{"id":8,"clabe":"0021800000000000001","titular":"Ana","saldo":540.00}

HTTP/1.1 200 OK
Content-Type: application/json

{"id":9,"clabe":"0021800000000000002","titular":"Beto","saldo":760.00}
```

Total en el sistema: $540.00 + $760.00 = $1,300.00

Se levanta transferencia con pausa de 20s y se lanza transferencia apagando cuenta a la mitad

```
PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia

# Enviar transferencia de 100 pesos y apagar cuenta en la ventana de pausa:
curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":100}' &
sleep 4
docker compose stop cuenta

HTTP/1.1 504 Gateway Timeout
Content-Type: application/json

{"path":"/api/transferencias","status":504,"error":"Gateway Timeout"}
```

Se reactiva cuenta y se consultan los saldos

```
docker compose start cuenta
curl -i http://localhost:8080/api/cuentas/0021800000000000001
curl -i http://localhost:8080/api/cuentas/0021800000000000002

{"id":8,"clabe":"0021800000000000001","titular":"Ana","saldo":440.00}
{"id":9,"clabe":"0021800000000000002","titular":"Beto","saldo":760.00}
```

Total en el sistema: $440.00 + $760.00 = $1,200.00 ($100 quedaron en el aire).

¿Por qué pasó?

Pasó porque al dividir el sistema en microservicios cada operación HTTP se confirma de forma autónoma en su propia base de datos; al no existir una transacción distribuida (@Transactional local ya no abarca múltiples servicios), si el cargo tiene éxito pero el abono falla, el dinero queda debitado sin que nadie lo devuelva a origin.

===> 2. Diagrama de estados de la Saga

Diagram (unsupported type)

```
stateDiagram-v2
    [*] --> INICIADA: Registrar transferencia
    INICIADA --> FALLIDA: Cargo rechazado (saldo insuficiente / 404)
    INICIADA --> CARGADA: Cargo exitoso en origen
    CARGADA --> COMPLETADA: Abono exitoso + registrar asientos
    CARGADA --> COMPENSADA: Abono falló y devolución tuvo éxito
    CARGADA --> COMPENSACION_PENDIENTE: Abono falló y devolución falló (cuenta caída)
    COMPENSACION_PENDIENTE --> COMPENSADA: Reintento programado exitoso
    COMPLETADA --> [*]
    COMPENSADA --> [*]
    FALLIDA --> [*]
```

===> 3. Código clave

• Entidad Transferencia y estados: Cada transferencia persiste su ciclo de vida (INICIADA, CARGADA, COMPLETADA, COMPENSADA, COMPENSACION_PENDIENTE).

• Orquestador en TransferenciaService: En transferir(), cada llamada guarda el estado en su propia transacción local. Si abonarADestino() lanza una excepción, se invoca compensar(t, motivo) para revertir el cargo mediante un abono a origen y registrar los asientos contables TRANSFERENCIA_ENVIADA y DEVOLUCION_TRANSFERENCIA.

• Idempotencia (Idempotency-Key): Cada paso manda una cabecera fija derivada de la saga (ej. transferencia-<id>-devolucion). Cuenta y movimiento registran la clave con restricción única para evitar cobros o devoluciones duplicadas ante reintentos.

• Circuit Breaker con Resilience4j (CuentaCliente): Las llamadas a cuenta se envuelven con CircuitBreaker. Si la tasa de fallas alcanza el 50% en una ventana de 4 llamadas, el circuito se abre y rechaza inmediatamente las peticiones sin sobrecargar a cuenta.

===> 4. La compensación funcionando

Usamos la variable de entorno FALLAR_ABONO_A_CLABE=002180000000000002 para forzar que el abono falle de manera simulada sin apagar los contenedores.

```
FALLAR_ABONO_A_CLABE=002180000000000002 docker compose up -d transferencia

curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":150}'

HTTP/1.1 502 Bad Gateway
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Transferencia no realizada",
  "status": 502,
  "detail": "No se pudo abonar a la CLABE 002180000000000002, asi que se devolvieron 150 a la CLABE
  002180000000000001: su saldo quedo como estaba.",
  "instance": "/transferencias",
  "transferencia": 19,
  "estado": "COMPENSADA",
  "motivo": "El abono a la CLABE 002180000000000002 fallo: Falla provocada con
  FALLAR_ABONO_A_CLABE=002180000000000002"
}
```

```
curl -s http://localhost:8080/api/cuentas/002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000002

{"id":8,"clabe":"002180000000000001","titular":"Ana","saldo":640.00}
{"id":9,"clabe":"002180000000000002","titular":"Beto","saldo":760.00}
```

```
curl -s "http://localhost:8080/api/movimientos?clabe=002180000000000001" | python3 -m json.tool | head -n 25

[
    {
        "id": 1346,
        "clabeCuenta": "002180000000000001",
        "tipo": "DEVOLUCION_TRANSFERENCIA",
        "monto": 150.0,
        "saldoResultante": 640.0,
        "fecha": "2026-11-02T00:53:56.215382Z",
        "referencia": "devolucion de la transferencia 19: no llego a 002180000000000002",
        "clave": "transferencia-19-asiento-devolucion"
    },
    {
        "id": 1345,
        "clabeCuenta": "002180000000000001",
        "tipo": "TRANSFERENCIA_ENVIADA",
        "monto": -150.0,
        "saldoResultante": 490.0,
        "fecha": "2026-11-02T00:53:56.199250Z",
        "referencia": "a 002180000000000002",
        "clave": "transferencia-19-asiento-cargo"
    }
]
```

===> 5. Cuando la compensación también falla (COMPENSACION_PENDIENTE)

Si durante la pausa de 20s apagamos cuenta, el abono falla y el intento inmediato de compensación también falla:

```
curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}' &
sleep 4
docker compose stop cuenta

HTTP/1.1 502 Bad Gateway
Content-Type: application/problem+json

{
  "title": "Transferencia no realizada",
  "status": 502,
  "detail": "No se pudo abonar a la CLABE 002180000000000002 y la devolucion de 50 a la CLABE
  002180000000000001 tampoco pudo hacerse todavia. El dinero salio de origen; el banco reintentara la
  devolucion automaticamente.",
  "transferencia": 20,
  "estado": "COMPENSACION_PENDIENTE"
}
```

```
curl -s "http://localhost:8080/api/transferencias?estado=COMPENSACION_PENDIENTE" | python3 -m json.tool

[
    {
        "id": 20,
        "claveOrigen": "002180000000000001",
        "claveDestino": "002180000000000002",
        "monto": 50.0,
        "estado": "COMPENSACION_PENDIENTE",
        "intentosDeCompensacion": 2
    }
]
```

```
docker compose start cuenta
# En los logs de transferencia:
docker compose logs transferencia | grep "Reintento programado"

transferencia-1  | 2026-11-02T00:56:54.313Z  INFO 1 --- m.m.transferencia.TransferenciaService   :
Transferencia 20: COMPENSADA
transferencia-1  | 2026-11-02T00:56:54.727Z  INFO 1 --- m.m.t.ReintentoDeCompensaciones          : Reintento
programado: 1 compensacion(es) pendiente(s) terminada(s)

curl -s http://localhost:8080/api/transferencias/20 | python3 -m json.tool

{
    "id": 20,
    "estado": "COMPENSADA",
    "saldoOrigenTrasCargo": 590.0,
    "intentosDeCompensacion": 7
}
```

¿Por qué la compensación debe ser idempotente?

Si la compensación no fuera idempotente, reintentar una devolución ante una pérdida de respuesta o timeout provocaría que el abono de compensación se aplicará múltiples veces en la cuenta de origen ("cobrar o devolver dos veces"). La cabecera Idempotency-Key garantiza que el saldo solo se incremente una sola vez sin importar cuántas veces se reintente la operación.

===> 6. Circuit Breaker: los tres estados evidenciados

Estado CLOSED (Circuito cerrado – operación normal)

```
docker compose exec transferencia wget -qO- http://localhost:8080/actuator/circuitbreakers

{"circuitBreakers":{"cuenta":{"failureRate":"0.0%","failureRateThreshold":"50.0%","bufferedCalls":4,"failedCal
ls":0,"state":"CLOSED"}}}
```

Estado OPEN (Circuito abierto – cuenta caída)

Se apaga cuenta (docker compose stop cuenta) y tras fallar las llamadas consecutivas:

```
docker compose exec transferencia wget -qO- http://localhost:8080/actuator/circuitbreakers

{"circuitBreakers":{"cuenta":{"failureRate":"100.0%","failureRateThreshold":"50.0%","bufferedCalls":2,"failedC
alls":2,"state":"OPEN"}}}
```

Una llamada recibida con el circuito abierto falla en solo 29 milisegundos sin sobrecargar a cuenta:

```
time curl -i -X POST http://localhost:8080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":10}'

HTTP/1.1 503 Service Unavailable
{"detail":"Circuito abierto hacia cuenta: fallo varias veces seguidas y no se le llama por unos
segundos","servicio":"cuenta"}

real 0m0.029s
```

Estado HALF_OPEN y retorno a CLOSED

Tras cumplirse los 10 segundos configuramos en CIRCUITO_ABIERTO_S=10, pasa automáticamente a HALF_OPEN. Al reencender cuenta, deja pasar las llamadas de prueba y regresa a CLOSED:

```
docker compose logs transferencia | grep "Circuito hacia cuenta" | tail -n 5

transferencia-1  | 2026-11-02T01:00:22.089Z  WARN 1 --- mx.mexibanco.clientes.CuentaCliente : Circuito hacia
cuenta: State transition from OPEN to HALF_OPEN
transferencia-1  | 2026-11-02T01:00:22.130Z  WARN 1 --- mx.mexibanco.clientes.CuentaCliente : Circuito hacia
cuenta: State transition from HALF_OPEN to OPEN
transferencia-1  | 2026-11-02T01:00:42.236Z  WARN 1 --- mx.mexibanco.clientes.CuentaCliente : Circuito hacia
cuenta: State transition from OPEN to HALF_OPEN
transferencia-1  | 2026-11-02T01:00:43.290Z  WARN 1 --- mx.mexibanco.clientes.CuentaCliente : Circuito hacia
cuenta: State transition from HALF_OPEN to CLOSED
```

===> 7. Lo que la Saga no te da (Aislamiento)

Las transacciones ACID garantizan el aislamiento (Isolation): ninguna transacción externa puede ver estados intermedios inconsistentes. En una saga orquestada, el aislamiento se pierde (ACID se convierte en BASE). Si un usuario o proceso consulta el saldo de la cuenta de origen en la cuenta de origen en la ventana de tiempo que transcurre entre la confirmación del cargo y la ejecución de la compensación, verá que el dinero ya no está en la cuenta (un dirty read a nivel de negocio), e incluso podría original termine siendo compensada y el dinero devuelto momentos después.